Lambda Architecture

systemdesignschool.io/fundamentals/lambda-architecture

We've covered batch processing (complete but slow) and stream processing (fast but handling late data is complex). Some systems need both properties simultaneously.

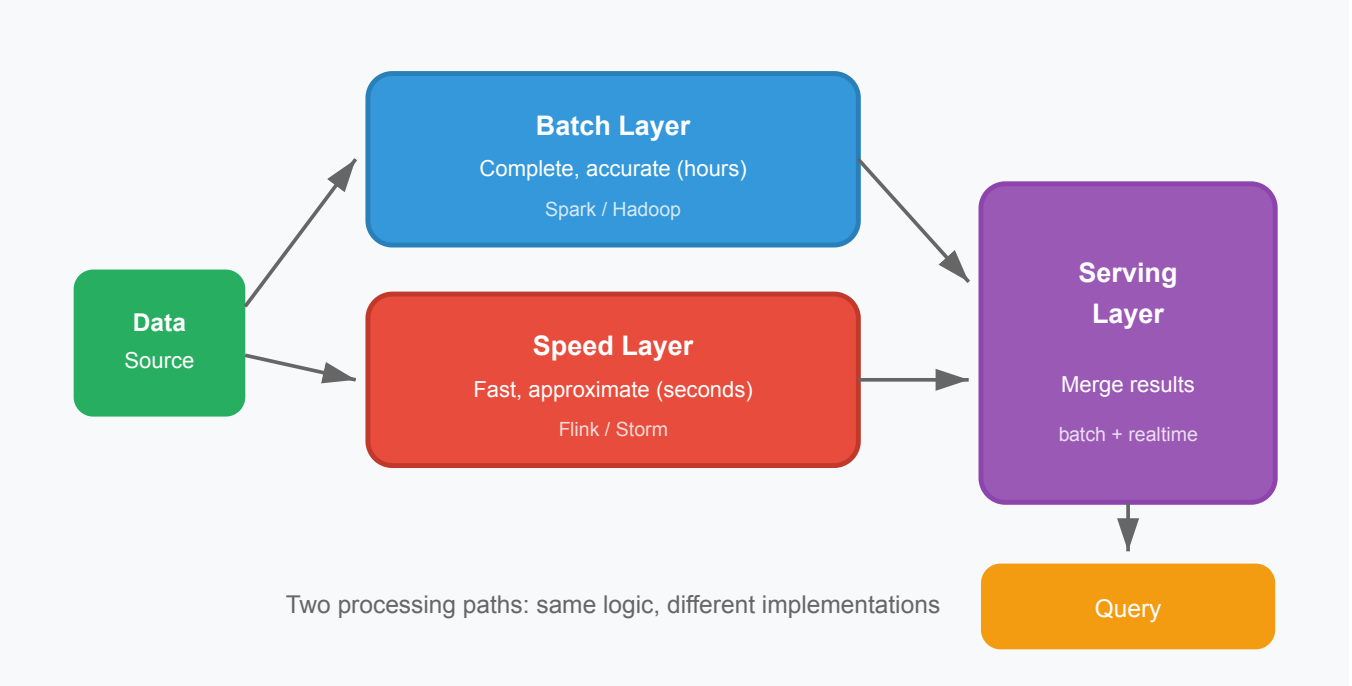
Consider an e-commerce company. The finance team needs daily sales reports accurate to the penny—they reconcile against payment processors, and discrepancies create accounting headaches. The operations team needs a live dashboard showing orders per minute—they need to spot outages immediately, not hours later.

Running everything through batch means the dashboard updates once per day—useless for incident response. Running everything through streaming risks missing late events or handling corrections incorrectly—problematic for financial reports.

Lambda architecture addresses this by running both approaches in parallel.

The Three Layers

Lambda architecture, popularized by Nathan Marz, organizes computation into three layers with distinct responsibilities.



Batch layer: Stores the complete, immutable master dataset (often in distributed storage like HDFS or S3) and periodically recomputes views from scratch. A nightly job reads all historical data, applies business logic, and writes results to batch views. This is slow—potentially hours—but produces complete, accurate results. Late-arriving events and corrections are naturally incorporated the next time the batch runs.

Speed layer (real-time layer): Processes events as they arrive via stream processing. It computes incremental updates covering only the time since the last batch completed. This is fast—seconds to minutes—but only captures events that have arrived so far. Events still in flight or delayed will be missing.

Serving layer: Merges results from both layers to answer queries. When a user requests "total sales today," the serving layer fetches the batch-computed result (sales through 2 AM when batch last ran) and the speed layer's delta (sales from 2 AM to now). The sum provides an up-to-date answer. When the next batch completes, its results replace the speed layer's estimates, and the speed layer resets.

Walking Through an Example

Let's trace how Lambda architecture computes total sales by product category.

Batch processing (2 AM daily):

1. Spark job reads all order events from the past year in S3
2. Aggregates sales by category: Electronics = $45.2M, Clothing = $28.1M, etc.
3. Writes results to a batch view table with timestamp "2024-01-15 02:00:00"

Stream processing (continuous):

1. Flink job consumes from Kafka order events topic
2. Maintains running totals since 2 AM in Redis: Electronics = $180K, Clothing = $95K
3. These totals only include events that arrived since 2 AM

Query handling (anytime):

1. User asks: "What are Electronics sales today?"
2. Serving layer fetches batch result: $45.2M (as of 2 AM)
3. Serving layer fetches speed layer delta: $180K (since 2 AM)
4. Returns: $45.38M

Next morning at 2 AM:

1. New batch job runs, computes Electronics = $45.4M including yesterday's full activity
2. Batch view updates with new timestamp
3. Speed layer resets counters to zero
4. Now queries combine $45.4M (batch) + new deltas (speed)

The Maintenance Problem

Lambda architecture achieves both speed and completeness, but at a cost.

Duplicate logic. The same business rules must be implemented twice—once in the batch system (perhaps Spark/SQL) and once in the stream system (perhaps Flink/Java). Subtle differences in how the two implementations handle edge cases lead to discrepancies. Users see different numbers depending on when they query relative to the batch run.

Dual infrastructure. You operate two separate data processing clusters with different technologies, deployment patterns, monitoring, and failure modes. When the batch layer has a bug, you fix it and rerun. When the speed layer has a bug, you fix it, redeploy, and may need to reconcile historical data.

Synchronization complexity. The serving layer must track which batch run is current, which speed layer data corresponds to that batch, and handle the handoff when a new batch completes. This coordination logic is error-prone.

When Lambda Makes Sense

Despite the overhead, Lambda architecture persists in scenarios where:

- **Compliance requires exact match with batch systems.** If audit systems or payment reconciliation use batch processing, the real-time layer must be seen as an approximation that the batch layer periodically "corrects."
- **Existing batch infrastructure is mature.** Organizations with significant investment in batch pipelines (Hadoop, data warehouses) can add a streaming layer incrementally rather than replacing everything.

- **Batch and stream genuinely have different semantics.** Some computations (like ML model training) inherently require full historical data and can't be incrementalized. Lambda lets you run batch-only logic alongside streaming.

What's Next

Lambda architecture's maintenance burden motivated the search for simpler alternatives. That leads into **Kappa architecture**—Jay Kreps' proposal to run everything through stream processing, using log replay to achieve batch-like capabilities without a separate batch layer.